Oh here are now, can't we?
What's an STTM? Source and uh source tag mapping this talk. What is a source target mapping?
No, no, no, transcript. You said yes, you said the last thing you said. Two things have happened since I did that. One, a closed laptop on which I was already playing the transcript.
You have to outsmart me. Is the transcript running? Yes. Have you closed the laptop on which it's running? Second question. This is your responsibility. Certainly not mine. Eh? We have one job. You have one job. It certainly isn't to learn anything. Learning on this course occurs purely by accident.
Transcripts, however, very deliberate. All we need is a record as if learning had taken place. That's what we're doing here. You know the Russian pretending villages.
When St. Catherine the Great was going down the river in Russia. And uh they wanted to uh uh convince Catherine the Great that Russia was great and wasn't having any famine or any uh stopping for starving, they built their fake villages along the river to make the Russia seem more prosperous.
That's what we're doing here. We're making a transcript as if learning has taken place. Try very hard to avoid it. Anyway, is this on recording on record now? And is there a transcript? Yeah, there appears to be a transcript. Okay. Source target mapping. What is a source of target mapping? An artifact that shows where the data comes from and where it's going to.
Kind of. Yeah. I mean, I mean the core of that artifact is what?
The stuff attributes on both sides. Thank you. Uh nope. It's the transformation rules. Transfer. Including transcripts. Right. So what kind of transformation rules? So let's say I've got a table over here. We can call it table, for example, I don't know, they call it A or B or something like that. You can make it realistic.
So there's there's my sources, A and B, and I want to join these sources together into let's call it T, the target. Yeah. And what source target mapping is is a series of rules that allow me to specify the target in terms of the two sources.

So for example, joining those table to tables together, right? But also, for example, what do I do to column to A to arrive at in in T. So for example, if I've got column, A1, what is it?
Well, where's A1 in T? Well, is it is it just it's just A1 in T, for example. So in other words, we could just have we have a check. And make us realistic if you like, but leave that track for now.
We could just say that the first column in A is the first column T. That would be one of the rules. Make sense?
Yeah? Not very complicated, I'm sure you all know. How many of you know these things? But we have to take it off. I have to take this thing off, right?
Remember? We're not here to learn. The transcript is to record as if we've learned, right? So we have to. Right. So some of you may many of you may know this already, but we haven't taken it off anyway.
What kinds of does anyone know the kinds of rules you can have when you specify columns in the target in terms of columns and the source? What kind of one of the name of the rules?
For example, direct is one of the rules. Direct is just where you take a column in the source and put it directly in the target. Anyone know the name of the other ones?
Direct. There's constant based by a constant. There's lookup. Where you look up in a third table. There's aggregate. Where you use a mean or something.
There's derive. Now derive is the main one. Derive is like transform, derive or transform. Derive or transform. That's where you apply an operation to the column.
What else? Basically, yeah. But there are uh people list other ones. Um people sometimes um break up derived into lots of different subtypes. They'll say for example, concatenate.
We have concatenated columns. Or they might say other things. Cool, like like you might say. Uh you might say uh concatenate someone, the bigger one, yeah. Uh okay, so direct constant lookup aggregate derive. If you type into catalyst, um

what are the types, what are the STTM or source tag mapping type column types?
It will give you a CG-specific catalyst specific list, which isn't exactly this list. I think derive is broken up into more types. However, um these are the sort of textbook ones.
I think one's in CG. There's a skill in catalyst. That can that that that will give you uh these sort of things. Now, okay, so that's so source tracket map.
What else do you have to do deal with? So when we're doing when we create data contract, right? Um we're specifying a data product. We have to specify a P I B product requirements document.
We have to specify a data contract.
Might include an additional glossary or a data dictionary, sometimes called data dictionary. Uh what else might we include? SLAs? Okay. Okay, I talk about SLAs. Slashly and slow, right? We know we know that. What do what are these stand for then? What's slars? Slide slide, yeah, anti-fly. Is that how you would say magnetic?
Slay. Well, yeah, he's done that. I have no idea what this is. Oh, okay. Source level agreement. What's uh what's a slide? I see. Indicator, thank you.
And what's a service level objective? There we go. Oh well. Um, so now we'll come back to uh SLAs in a second. SLIs, what do we what do what's the definition of an SLI?
What are SLIs? So they're the measurable quantity, but the SLA will be bad. So the indicator, for example, an indicator could be delayed. Alright?
Delay? Yeah? Not in an agreement? Don't care if you understand what the word delay means. Don't care if you learn anything. I care that you're not in agreement, so we can all pretend that learns.
That's what our goal is. So we've got delay. So uh what's a service level objective? The actual time by which you would start nodding a delay as occurring.
Uh it could be, it's the ideal value of a delay. But you get kind of but they actually

do want to be similar things. It's way well, it's ideal value.
Zero would be ideal value, right? Uh this SLA is what? It's almost like a um like a level of service uh agreement. Yeah. But if we were to define it rather than repeat the uh initials, uh we would say that it's the permissible departure from the objective.
So uh delay is allowed to be an hour or something, right? But over an hour you're breaking the agreement, yeah. Okay, good. Can we list some uh can we list some SLIs?
So what kinds of things, what kind of things are service level indicators? What delay? What else do we have? Can you say that louder? Uh is that an in uh is that an indicator I think delay is just zero, right?
That would be something on time would be delay of zero. So for example, things I want I want a list that includes things like latency. I want recovery time.
Can someone give me some others? Three put, yeah, latency. Throughput. Uh recovery time. Well in delivery time as well. Delivery time, yeah, yeah, capital time, maybe.
Well, I'd I maybe think about that as delay from delivery time, just because the objective is hard to define the objective. You say the objective is 7am.
What does it really mean? It's like like a microsecond away and then you're breaking. So you're delayed from delivery time, but yeah. Which time zone and all those other things?
Yep, time zone, yeah. So we we uh so we're trying to measure here's the thing that we're trying to measure. So we've got latency, throughput, recovery time, deliver delivery time or delay from delivery time.
What other things? Probably most of them. Completeness. Complete and good data quality measures, right? Data quality measures, EP measures, completeness, accuracy, pressioness, all that all those other things, right?
Good. Um products have you may not call it this, but they have uh support uh measures. Um so for example, some products might have a duty to explain.
What's a duty to explain? Well, if you get a uh if you get a piece of data through

that appears to be incorrect, um there might be an SLA or something of measuring around how fast a turnaround time is for someone to explain that, why it's interactive.
So for example, if you're looking at, for example, I don't know, an oil futures market, and then you're getting prices from some vendor, and then negative prices, you're gonna call someone up and say, why am I all futures negative? Right? Why is the future why is the future price of all negative, right? Anyone know why the future price world went negative? When did that? COVID, right? Because future contract you have to have to take delivery of oil. So if so if you if you can't take delivery of oil, you're paying other people for the contract.
So there's happened to be uh that's one of the rare cases where the future price of all went negative. There's a very famous case in London, actually, of oil future traders.
It happened, I think in the 90s or something, and they sent barges of oil down the river tens to the actual stock office because they have to take delivery.
So uh cool. So there's the thing that uh there's basically support uh ones, you can think of that support, you can think of that as okay, right?
So recovery time, what's recovery time? So it's back online, or it could be service to a certain level of like um data recovery, for example, or service availability.
So we haven't put availability back here. What's availability? Pardon? Yeah, maybe we each can be reached, yeah, it's because of service work. Uh uh, how how what percentage of the time is the service working?
Yeah. So typically we make a uh measure only these things in percentiles of time. For example, availability we measure in percentiles of time, and then this is the last thing.
So times it is in nines, right? So we've got one nine is 90%. Two nines is nine. Three nines is like nine point nine, four nines, and then five nines is usually where we stop.
Okay, so what percentage of a year is one nine give me in days. How many days is is is ninety percent of the year? How many days is 10% of the year?

Yeah, two years. 36 days, right? So the so if our service availability is 90%, we can be away about a month. Cool, cool. Because it's 36 days, right?
I'm gonna say back number, right? Uh two nines. How many days or hours or whatever of a year is two nines? Nine percent of the year. How long can we be down?
3.65. Yeah, so I call that a working week. Working week. Three nines? Eight hours. Eight hours working day. Working day. Four nines? One hour. Hour?
Five nines? Five minutes. Five minutes, yeah. I I I just think of this like a minute. I go down by month, week, day, out. So if you wanted like a rough rough sense of how long can you be down, well, one nine you can be down a month or a year. Five nines can be down about a minute. Well, how will can we reach like one hour in terms of like you know, national production? Sorry, how many services?
Oh, but I've talked about that, so that's good. Um so classically in software engineering, right? Classically software engineering, uh, most global scale businesses would be offering four nines of availability.
Right? So nuclear power plants would be offering. Actually, nuclear power plants are varying actually. Energy power plants usually offer somewhere between three nines and six nines of availability depending on what kind of power plant they have, right?
How many nines does anthropics claude have? Do we think how many nines of availability? One. How many nines is GitHub have?
Before Microsoft purchased GitHub, how many nines did GitHub have? Less than so classically, before in the in the before times in software engineering, you would think that most services would offer.
I think AWS, for example, I don't know if they typically offer four lines or three nines. What it really means is that they may be down for longer than that, actually, not usually.
They may be down, but they would then give you free credits if they're down, or they would they would then pay for support and other things, right?
So there's um there's one kind of recent case where Amazon lost data and was

down for more than more than the usual three, four nines, and that was because of a uh bombing uh bombing in the data center.
So they went and trying to find all these things. That's what it takes. That's what it takes to take out on Amazon for the sample time. What does it take to take down GitHub?
An employee arrives at Microsoft and just size in what does it take to take down Amazon uh bombing by MRN, right? Different levels of service politics.
Um okay. What about their crowd uh the crowd strike thing at item? Uh I don't think crowd strike push down AWS, you can tell me CrowdStrike is actually windows on top, right?
Crowd strike was a let's talk about different things. Uh yes. Okay. So what is a realistic number of lines? I don't know, but uh more than one. Right?
So if you're providing a service and you're down for more than a working day a year, it's not a very reliable service to be honest. How do we as a final point? How do we how do you increase the number of lines that we're available for? How do we actually stock engineering terms or disaster recovery? So in general, we we clone our infrastructure, right?
So you've got some infrastructure over here in data center one, and then we clone our infrastructure in data center two, and then we clone as much infrastructure as we want to, and then we have balancing systems in the front, right?
And then we you know, maybe we we send 95% of our systems over here, we send maybe 2% over here, and we send another 2% somewhere else in the world, in order that we are constantly checking that our disaster recovery systems work.
We don't we make sure that some lie of traffic goes there. And then during the disaster we switch over between the different data centers. Right?
So if it's global, this is called global load balancer. And you can also have local load balances as well within data centers. Um the reason that Amazon is particularly proficient at uh having many nines is they've got many identical

cloned data centers.
So even on one park or one data set that you've got many uh reproducible units.
So it's quite easy for Amazon just to fail over to a different data center.
So you actually have to bomb a very large region for anything to actually be lost.
And so you ask yourself why are, for example, clawed and other things, why are they not alike the same level of performance?
Well, the answer must be that they're at tremendous they're at capacity. They're using if they're not doing 2% over here, they're doing like 100% over here, 9% over there.
They don't have a lot of spectacle if any. So these nines, they tell you basically how much spec capacity the provider has. Um for most large providers, the number of nines that they have tells you about their capacity rather than about uh their set of quality in the stuff.
Okay, so why are we talking about all this? Well, you have to know it. Um what we'll do this morning. Um, what we're gonna do today is I want you to, in some groups, just go through all that and add those to your data contracts or PIVs that you're doing yesterday.
Cool, just the SSTMs and all that, half an hour, whatever. Um and then we're gonna talk about PagePath, we'll talk about Docker and all the technologies of the page path, and then we'll do some technical challenges, we'll talk about the dev containers, we'll do all that.
Cool. So here's what the challenge is going to be. Um I want you to ask your catalyst individually, um, what types of STTM transformations are there?
What type of SLOs, SLIs and things are there? Cool? Just see what happens.
Here we get that's individually. And then in on your tables, maybe, or whatever.
Um, I just want you to, or maybe two to threes. What do we want? Cables, two to threes. People want tables because it allows them to do nothing.
Yeah. I don't really care. That's just tables. Um on your tables, I want you to come up with so the couple of the data products you were working on yesterday in terms of the PRDs you generated and the data contracts you generated.

I just want you to have a discussion with some people around you on your PRDs or any of the specifically under data contract actually, rather than your PODs. Um, what are the incoming tables? What's the outgoing table? What are some example rules that you want to have? You can transcribe this, transcribe the conversation, then you can just like Apple Memos, right?
Then you can drop it into your copilot, and I want you to uh extend your data contract with realistic STTM for your product and some SLAs, some realistic SIs.
Cool. Right, so it's 20 past nine now. Um should we come back at 10? And I'll give you a break. You can just 20 minutes and then you have 20 minute break or something.
25 minutes, pretty break. We'll see what time. I guess. So it's like 20 number.
Yeah, I mean, just sort of keep going until you feel like you have a good STTM and you've I mean 15 minutes, you know, discuss it with people.
So if you many of you will have different data products coming from different tables because you're in different threes. So maybe you just choose a couple of data contracts you have.
Um have a discussion around what the incoming tables are, what the outgoing table is, what some of the rules would be, really detailed discussions, like we're gonna have to do the we're gonna have to, for example, um look this up in some of the third table, we're gonna have to apply this rule to this, we're gonna have to apply transformation or whatever.
Trying to transcribe that, but I want to see a couple of data contracts with FTTMs and some SLAs that are actually realistic for the data product.
Cool? So give yourself like 20 minutes, 25 minutes or whatever it takes, and then the rest of the time you can have for a break, and we'll see you at 10.
And you'll need coffee, you'll obviously need coffee. And can I ask you to have some fun while you do that, please? Just five minutes, don't overdose it.
I can see some of you overdose on fun. But just five minutes of fun.
We all here.
So who's Saravana from group five? Where are we? Okay. We can have some

merch for your submission. Sure, why not? Why not? Catch. Yeah, hear it.
Oh, no. Oh, I want it to see. Thank you for a while.
You've got one? Sure, why not? Thank you. Oh I thought. Um, right. So let's have a look at this one briefly. Uh we've got um now now this of course is a little hard to read.
A couple of things we can do, we can open preview. Yeah, you can open preview. One thing we can do maybe is open with markdown editor. If we do that, it's like opening preview, but we can hopefully press the edit button and edit it.
My data contract from group five. Uh and group five is evil. So we go. Cool. Yeah, it's here, no evil seen no, that's the three monkeys, right? Here, no evil seen. That's one. Alright, um, okay. So there's the editor. Now, if we want to share this or we want a better documentation for our project, what else could we do?
Cool. We could ask our best friend in the world. Opus. And you could say, um, I don't know what should we say. Create a well structured HTML document uh with appropriate visuals, diagrams, and other helpful assets from this file here.
What else do we say? Add pros, add simple English prose to explain areas without any guidance. Um create a good can I create an overview at the top, a table of contents, I make it easy to read and now they go.
Uh uh create the HTML file self-contained. There you go. Um you want to change log in there too. Do you want to change log in there too? I think there's a change over the bottom anyway, isn't there?
Oh, okay. Oh, look at that. Yeah. I mean this these things have never happened, right? Okay. So um, yeah. I guess what I'm trying to do here is more I guess show you the mark and editor, which would be helpful for you because when we look at the captioned event, for example, or other challenges on Wednesday and Thursday, there's lots of markdown files, and if you open the editor, um it's a little easier to navigate than the preview.
Also, if you're sharing things via your teams, or you're showing things with each other, you're putting them in a git repository or something, you can put the raw markdown in there, or you can put the raw raw markdown in a file somewhere

and actually convert it to HTML and I'll put it on there.
I mean from an AI point of view, this operating on markdown is not very basically the same as operating on the HTML. So that the there's no loss of power that Paul is gonna have uh from that.
Now one of the downsides of using the fattest model with the highest context and the most reasoning is it's slowest, right? We know that we know that from our lives, right?
We know that from our minds. Anyway, I don't need to consider what you should do. This is all the background reasoning we've been talking about, right?
That's what it's doing now and generating best amount of background tokens that we can't see. We used to be able to see them, but we stopped do you know anyone know why we stopped being able to see them?
Or what if I'm doing talking about? Any other any other things while we're waiting? I should have chosen a small model, but never mind. It's more appealing to look at.
I thought you're gonna be able to do that. No, I see a fake summary, it's not the real story, you've got real reason to people kept interrupting it, and oh never took it.
Because um competitor labs, competitor vendors would take those reasoning traces and use it to train around models. And I'm told I'm until very recently the way around the way of cheating around it.
Basically what you would do is you would ask models to complete conversation. Remember how I said conversation was structured in the training data, you would have prompt completion pool, reasoning pool, etc.
But what you would do is rather than giving the models just the prompt, you would structure the conversation to all missing bits. And if you put the if you put a missing reasoning bit, the models would fill it out with your actual reasoning.
You could kind of cheat them on a cheat cheap this notation. What's going on? Okay. Well, this was gonna be a nice demo, wasn't it? This is why you start this is why this is when you start swearing, right?

What are you doing? No, I don't let it think. Beat it until it's done. No, no, no, no, no, no, no, no, no, no, no, no. No, you don't you don't create the future with patience. So you you enjoy the past with patience. You create the future with transient contempt for time wasting. Exactly. Well, oh well. It would uh the thing is it would be I can't even tell if this is even doing anything.

Yeah. So what I'll do is what I'll do is I'll show you what the difference is with a smaller model and the what I'll do is I'll go down to Luna.

We'll go to low effort. We'll paste the same thing in. And already done. Well it started anyway. Oh, so that's not a reasoning. No, no, no, no, no.

This is a summary of the reasoning. This is something that they send you to. Let's have a look at the other one. Is it still there? Okay. Gone further, apparently. 43 lines. Okay. Well, this is going to be an SMA wasn't it?

But it's been ruined by um Sam and Dario. The CEOs of OpenAI and Anthropic. Sam is uh psychopath. And uh Dario, Dario is hard to read. I think it's very delusional.

Okay, so Luna is finished, there we go. So this is the this is a cheap fact one. Um can we open a preview? Can you open an integrated browser anyway?

There you go. So this is something, isn't it? Temple contents, easy to browse through. Uh so it's done a little bit, right? It's done a little bit.

It's definitely sort of better, easy to navigate, isn't it, than the markdown? And that's self-contained. And we can iterate on that file. So that's using Luna.

And later on, when Opus is done, I'll show you what's right. I imagine it's gone going and generating lots of SVG diagrams there. Um, okay. So my point there was to show you that you can ask for HTML uh versions and you can iterate on the visual quality of those versions using AIs, you can add diagrams, you can add other things for the purpose of documentation.

Alright. Right, let's talk about the paid path then. So uh so we've done that's kind of draw the line under the design of data products. Uh I guess there's one or two more things to say about STPM.

And then that draws the line. Um so when we're specifying STTMs in the design

phase, this of course is all in English, isn't it? So there's a gap between this and what the data product creator will necessarily do, or what you'll do when you come to crypt a thing.
Um so one of the what are some difficulties in specifying rules English? In other words, what are some uh problems we might want to address in the design of this document before it goes to development phase?
What's the what's some problems with specifying rules in English? Can you take us?
Yeah.
Yep. So well-defined things, ambiguity. Edge cases. Ambiguity in edge cases. What are some edge case values, that common values that are an issue?
What's one of the most common values, which is hard to null? Nulls, yeah, etc.
Yeah, so you have to define what null means in a particular column.
So null is relative to the column, right? So what's the word null means? Um zeros, maximum values, minimum values. So we know for data quality purposes, we would know what the maximum minimum value is, etc.
Cool. So that's part of the dev that's part of the uh resolving ambiguities. So when we're iterating on a design document, we only need to make sure that all those ambiguities are resolved at design time, yeah.
Education is ambiguous. Alright, let's talk about the pay path. So the pay path is a set technologies, isn't it? For creating data products. Does anyone know what set technologies are on the paid path?
Adericks, good. DBT, good. Any others? Remo. Nope. Doctor. Docker, yeah. Airflow, yeah. Airflow Stromer, we can put a flash there. Any others? Catalyst goes on there.
S3. S3, yeah, we can put that down anyway. Uh databricks, speak for S3. Yeah, you need to catalogue a database part databrics, but yes. Any others?
I think that's it really. Okay, so on the paid path. Uh what does DBT do? What's DBT provide for paper? Transportation. Transformation test the STM rules.
Cool. Data product. Quality, yeah, yeah. So it provide DBT provides two things. It

provides your transformation rules. And you provide the data quality tests.
So you write your SETMs in DBT and you write tests for your data quality. I'll show you some of this in a second, so I don't define them. Um what does Databricks provide?
This is a tricky question. Provides the compute. So all Dabric is doing is actually running the DBT. So DBT is a language, think about SQL. It is very simple to SQL. Um what runs DAT is database? Cool. What is S3 provide? Storage. Yeah. So S3 is a cloud file system. And we'll talk about that. So we'll talk about this.
Um so we've got storage REST. It's a cloud file system. So our tables are stored in S3. Right? CSV, then on that part. Uh well, parquet file. We actually want to go into more detail with these, but yeah, I agree.
Okay, we can do a little sidebar now. So it's probably it's actually delta files. Is it this paper path file? Written as a delta symbol. We talk about all this in a second, but I'm gonna give you the high level first and then.
So we've got S3 for storage, data rates for compute. Uh DBT is the language. Yeah. Uh what's F for? Workflows. Orchestration, yeah. What is the hell does that mean?
Scheduling. Scheduling, uh, what does that mean? So time, yeah, the timing is a bit more specific, I suppose. It's a timing and triggering of events.
The key events for us are of course just running dbt. Cool. So airflow is the system for specify when what events will cause dbt to run. Cool. What's astronomer?
Uh yeah, you can think of it just as a UI, but it kind of isn't. Manage Airflow. It's managed airflow, yeah. So astronomer is cloud airflow. What does that even mean?
Right. It means that if we didn't have astronomer, we would have to run Airflow ourselves on Amazon, for example, right? And we would have to have machines and we would manage those machines.
What astronomer is strong is a vendor of Airflow, and they're managing Airflow running for you. Um why doesn't CG just, for example, run Airflow themselves?

Well, actually, you do. That's something that's off the page path. Well, roughly, right? The page path is moving to manage airflow. Why? Because I'll talk about this in a second as well.
It just pushes out the ability for you in your own team to manage Airflow rather than go through a central platform. Because you can like render, you can essentially manage your own airflow instances by going to the vendor of that system and saying, spinning up an airflow instance or something.
Rather than having to go through a central platform team. So CG was running Airflow on its own infrastructure. We have to go by a central platform.
So they're handling, they're handling that in four cases. Cool. Okay, so we've got airflow for scheduling. We've got a database compute, S3 for storage, GBT for the query language.
What's buckle for? Containers. Containers. Tell me about that. What's that? Yeah, so you can think of Docker as wrapping up your code into a reusable or containerized modular unit.
And we can deploy Docker. So when we're developing locally, for example, or we can develop locally, and then we can deploy those containers to whatever we're running them.
We'll talk about that one. Renewable catalyst is what role does Git play within the system? What is git? Code storage control. Code storage, yeah, so version control for code.
It has an operational use here. PRs and things, what is that? CICA, blah, blah, blah, yes, okay. So in other words, when you version your code, that's an event within a system, in fact.
And actually you can tie things to that event, such as releasing code. Right? Now there's a few intermediate sets which we talk about, but basically, one of the roles that git and GitHub will play in the system is triggering the release of your DBT and network mode to live environments.
They're harnessed for the background will do that, yeah. But you have the data creators work that's my trend product set up for you. Uh but um the key that

yeah, the sort of the trick that makes the whole thing simple simple enough from a data creator point of view, is that you can write DBT and Airflow, and there's a whole system behind the scenes I can get, which would enable the deployment release of your system of code without having to worry too much about the comments.
Cool. Alright.
Any questions? Say that ladder. Uh what about DLT and like all command? So I can I can hear the phrase ELT. DLT. DLG. And late flow coming. Don't know.
So late connect is uh connecting the S3 backend storage to a API stuff for like producing things as APIs and let's go away if we can ingest as a data thing.
And DLT, it's uh basically dbt file main to call data. So dbt is the well, dbt is the pay path query language, I can say. The other thing is um I think this pay path isn't for um first of all, you don't have to be on the pay path.
You have permission to use pay path, if you have a real need. One of the cases, which I think you thought you talked about, is what if we want to bypass effectively data bricks and have like or bypass the querying layer here and serve an API from you know from S3 or something like that.
That's all that's permissible, that would be part of what we're doing. Uh from drag Dremio connection to data breaks, so you see you injusting data in a different system that we talk about.
Oh, like uh instead of in data from data from a bit. Yep. And yes, and they're gonna be from the so there's there's two different this is a side conversation which we'll be going to, but there's two different types of text tables that you've tried.
So there's a management manage. The paved path would be ones where there would be an S3, so the B manage. Um if Datricks has the tables, that's fine, that wouldn't be part of paper.
We can talk about why that is. Question? Yeah, so I have a question about the page panel of the small person here, you know. Yeah. Is this like the philosophy of SCMs?

Yeah, so the page, you know where we were listening to Hero at the beginning? He was here yesterday or something. I'm sorry, I was I was a layers.
I was uh last one here, so I definitely asked that. So how expensive it is for hero to be anywhere in the world. Every feat we move them away from his office, it's like thousands of dollars, I'm sure.
Yeah, exactly. So there you are. Okay, okay. Uh so uh, yes. So the pay path, there's I guess there's I don't know how many uh phrases there's data product if you like, apart this course, which is the everything we've been talking about, and then there's a page path, which is how do you what's the actual technology behind crane data products?
This is the standardized set of technologies that are the recommended set I'll just crack data, right? Um now, as I just been saying, you can leave the page path if you have a specific use case.
I'm my job here is to teach you about the pay path. Okay. Um there's a whole world of like other things, especially with pay bricks. Databricks has got its own scheduler, for example, real-time stuff, the things for software engineers, lots of things for software engineers, or people's software engineers all very interested in.
Yeah, that's the paid path is answering one to say question, which is just how do you create data product? Remember, data props are discoverable, they're searchable, they're all these other things.
That wouldn't necessarily pile engineering cases. So you if you've got like for example, app integration between two different systems, that's not a data product.
Uh because um it's not the discoverable data asset within the okay. So we want to get into these things a little bit more. Uh what I'll do is um what we're gonna do is tackle some of these uh um at slightly deeper levels as we go throughout the day.
Um what I want to get to is by lunchtime I'm gonna get two sort of Docker, and then I'm gonna give you some challenges associated with Docker in the dev

container.
Um I guess I want to talk about Databricks now. I want to build up to Databricks, and then I want to make a break. Cool. Let me draw a diagram for you though. So what are all these technologies add up to? So we say the page path, what are we really saying? Well, we're saying that there is a remote file system called S3. Here's S3. Here's some table files on S3. We could draw them as Delta, it's a delta symbol. There's like a CSV file or something. There's one. And that's coming in, and that's dropped in, let's say that's dropped in there from Morningstar as a CSV file or whatever parking file, whatever.
Um, and what what our job is as a J product creator is to transform that file into some final file, consumer aligned file, right? And the way we do that is by applying transformation rules until we produce the final file.
Yes? Make sense? So transformation rules are written in DBT. They run on data breaks. And they're triggered by anyone? Airflow. Cool. So the actual operator in line operations that we're interested in is relatively simple.
We just start with a file, the table. Uh copy and pasting it almost. Until it's different. And how do we apply those transfer? How do we write that with the DBT?
Do I have a query running on database? What triggers it up? Okay. People uh some people look uncertain. So we've all look bored. So I have no idea how to do that.
But if I take the average of those two faces, we're all good. Right, uh, hopefully everything will become clear as we go through the technologies.
So let's work there. Yeah. Yeah, remember, not here to learn anything. The transcripts are, right? Correct. Okay. Otherwise, there'll be no point to this.
That's the our our point is to create the perfect transcript. Okay, good. So let's let's look up to our user data bricks. So these delta files and why we're doing all this.
But to do that, let's let's just think about data systems a little bit. So I want to talk about data systems, right? Data systems. Uh let's start with data stores.

Anyone define what data store is? We're gonna find some words here. We're gonna define data store, database, data warehouse, data lake, data lake house, and then when we have a data lake house, we'll know why we're using data breaks.
We take a database of providing us with a data lake house. Okay, so we're gonna build that too. So once we want to define those who are desperate for coughing release you into the while.
Uh what are uh what are data stores? Data store, yeah. What's a file uh another word for a data store is a file system. Data store. Uh S3 is a cloud file system, isn't it?
File system. What characterizes a file system or a data store? What does the data have in common on a file system? No, nothing, that's nothing in common.
I mean, there's one fact you can say about it on a modern machine, which is written in bytes called binaries, maybe zero than one, but the other otherwise you have no common structure.
In fact, when file systems were originally invented, let's say 1950s, roughly, um you couldn't even say that they would have that in common. Because some computers were on decibel.
Some computers even had like three three states, some are binary. Uh some computers, anyway, we can talk about different differences in computers in 1950s, but they weren't, they didn't even have that in common.
So a file, so data store is a place where you can put basically data that has no common structure, right? That's why data stores are very useful.
Cool? So we go back to 1956, we have data stores. Now the problem with the data store is that there's nothing that you can know that these things have in common, which means that your querying layer, whenever you're accessing things in a data store or a file system, um, your querying layer has to know about the thing being accessed.
So your query becomes more complicated. Make sense? Make sense, everyone? Yeah. So in order to deal with the complexity of querying on a file system, um, we

invent.
What is that complexity? That's that's writing, that's awful engineers writing programs that know a lot about how to read files and so on. So to simplify that, we in we invent something, it's called a database.
What's a database? A large store system. Yeah, but compared to a data store, a structure. Common structure. So the known common structure, right?
Now there's a specific specific kind of database, specific kind of database that we all are interested in. What's that? SQL. SQL or relational relational database.
Okay, so what does the word relation mean? Ah, perfect! Oh no, almost no one knows this. EA know everything. Ah, well, thank you. Uh so people think the word relation has something to do with a relationship, it does not.
Relation means table. So relation is a mathematical word for table. In particular, if you don't know what the actual mathematical thing is, a relation in mathematics is a set of tuples.
Which is a set, which is unordered, collection of an unordered collection of ordered things. What does that mean? It's an unordered collection of rows.
So relation is specifically something where when you have a relational database, you have a table which doesn't know the order of rows, and the rows themselves have ordered columns.
That's why you have to have like primary keys in this. Because if you didn't you think, why do we need even need primary keys? Doesn't the database know it's one, two, three?
No, actually, it stores them in its own order, you don't know the order, not that kind of order. So it will store them in optimally. And then you can use primary key status.
Cool. So the mathematical definition of a relation is a set of tuples or an or an unordered collection of order things. Okay, so table. Okay, um, what do we want to say about that?
So we're in sort of 1970. Now this is software, we are 1970. Now the now relational databases have some nice properties. What nice properties relational

databases have?
Acid, right? Does anyone know what acid stands for? Atomic consistent. Isolated, but maybe durability. Durability, right. So let me give you uh, I think this is kind of uh I don't know, I I don't like the way that that is covered.
There's another property that all these things, we add all these things up together, they give you one property, which I think is the easier property to understand.
It's called linearization or linearizability, linearization. Let me explain what linearization is. Linearization is here's my database. And here's all the people around the world who want to write or read from my database.
Alice, Bob, Eve, etc. Right? Alice, somewhere in the world, writes one. And then according to reality, physics, the real world, Bob somewhere in the world writes two, and somewhere else in the real world later on the day, uh, Eve writes three. Make sense? Now, what we want to see in the database is one to three, regardless of what happens, between Alice, Eve and Bob, and regardless of what happens over here.
In other words, if the database is off or fails, or for some reason Bob is closer than Alice to the database and Eve, therefore it has a shorter cable, or if Bob disappears for a certain period of time.
If in the world we we write one, two, three, better, we better read out of the database one, two, three. That's very, very hard thing to do, because cable can have different lengths.
One can rate another person, things can fail over here, things can fail over there. It's a very hard thing to do. The way, by the way, the way that you get this property is that basically there's logs behind lots of these transactions, and the database itself is in some sense a reconciliation of many logs.
That's how the system provides this property is this intermediate logging of everything I have. So even if things that arrive out of order, the database will eventually be nice.
Okay, now it turns out if data data systems do not have this property, what we

found out, though many, and many people knew, we found out that they're basically useless.
However, we did go through about a decade if you were there. I was there. You know the L C D sound system, uh losing my editing. Anyway. Anyway, let's go with this.
I just get distracted by uh lyrics from 90s something. Uh I was there, so um. There was a wave of something called non-relational databases, non relational, right?
What when what are non-relational databases? Well, there's a clue, right? I just told you what the word relation means, so what is non-relation mean?
No, almost. This is data files? Could be, so there could be data stores. They're actually non data stores. They're somewhere between uh relational and non-related and uh and data stores.
So non relational databases are those which do because they're a database, not a data store, they're a database, so it does have a common structure.
It's just not a tabular structure. Right, let me give you some examples. Yeah, do you know JSON? Everyone knows JSON in the room? Yep. Right. JSON, if you have a few JSON files together, that is a non relational database.
Okay. Um few spreadsheets, Excel spreadsheets together. It's a non-relational database. Right? Now there's lots of different kinds. JSON-based ones of them, so if you like my pocket there.
What they're what they all do is they all, at least historically, promise to be much faster than the relation databases, because they had much weaker structural guarantees, which allowed them to be much faster.
For example, if I have a JSON document, in a JSON document, I've got many tables drawn together. You want to understand that? Like if I if I go to a JSON document and I say credit file, someone's city, someone's address, someone's location, I can get the location alongside their credit file, loan history, first loan, value of the first one.
All in the same file. It's kind of pre-joined. It's much faster. So non-relational

databases are those that are databases, so they have a common structure to their data, but they're non-relational, they're not tabular.
Now, most of those broke ACID, they broke linearization. Basically, um the way that they were able to be faster is by not being linearizable. Now, a lot of people went crazy for this, remember big data to create big data.
So what big data was mostly about is trying to apply non-relational databases to solve data problems. And people spent like a decade doing that, right?
And it turns out they were all basically useless. Because you lose data, things come out of order. It turns out for almost all real cases, you want linearization. You want this strong property that your database makes sense. Your database doesn't make sense. How useful is it? It's faster, but the data quality is much lower.
Right? So why am I talking about this? Well, good to know. Let's now talk about the cloud. Because these are the story of like databases from like 60s to 2000s. So in the early 2000s, something happens and we move to the cloud. What happens in the early 2000s? Dot com bubble. Dot com bubble, yeah. So from a stock point of view.
From a uh but the underlying technology there is the internet. Yeah, so we have the internet blow up, so we have lots of large amounts of data coming in, and the amount of data we have, the volume of that data is larger than the hard drives we have in single machines.
So if I remember my first machine in the Windows 98 machine, the four gigabyte hard drive, which I assume was an expensive hard drive, I don't know, right?
But I know that the computer's about a thousand pounds and 999 or something, that must be a lot of money, I assume. Um, the internet, you know, you you could have um many gigabytes of data just from your uh a million customers and something, right?
So the amount of data you're getting is is more than the hard drive. It's also coming in faster than the hard drive and spin very slow. So what we do instead is rather than using one machine, we use many machines, therefore we have data

centers, therefore we have distributed machines, and therefore we have a real problem, which is that to process data on a large number of machines is very hard.
Process a large amount of data on a large number of machines is very hard. So in response to the uh internet, we go back in time, we abandon databases, or we banned the databases, and we go back to file systems.
And the reason we do that is because a file system is a kind of minimum requirement of structure, it's a simple interface that allows us to bring together a very large number of machines and do arbitrary things with them, like putting data putting data on lots of them.
It's actually much hard. You think about if you think about a racial database, it's there's a level of inflexibility to a relational database. If you try and make a relational database in large like 100 machines, it's very hard.
Think about doing think about what I say about asset linear linearization, right? You've got all these machines in a relational database making up of one relational database, a hundred machines.
Think about the network connections between them. Think about the amount of logging you would need to do to preserve the quality of the data coming in because you could have all kinds of uh failures happen right within that system.
Anyway, it's just that's fine. The key point is that we uh we invent a cloud file system. Now, S3 is the you know one of the key ones, right? Cloud file system.
Now the problem with their file system, remember we go back to the 1960s? Problem with the file system is what? Problem with the file system from a querying point of view.
You don't know the structure. In general, we assume there's no common structure. So that means our querying layer has to be very complicated, because the querying layer now needs to impart the structure that we don't know the data layout.
Does anyone know what the kind of first major cloud file system query engine is? Maybe you heard the word Hadoop? Many of you have heard the word to do, so

that's good.
So Hado is kind of these two things. It's something called HDFS, which is a cloud file system, plus, which is a querying system for that file system.
Either way, the point is what Hadoop invents in the mid-2000s is a querying approach for a file system. Okay. Now we know, because we were around three minutes ago when I said this, so we know now, right?
We know that a data system that does not have this linearization property is useless. Who cares about having data if the data quality is now you don't know what things, what time things are coming out, you might have lost something because something is dropped down, the logs aren't accurate, right?
Who needs data that isn't that doesn't have quality, right? So we go through several waves of fixing Hadoop. What is the biggest kind of competitor at who that who begins to start fixing Hadoop?
We're almost, I'll give you a clue. We're almost ready for a break. So there must so if we're almost ready for a break, right? Towards the end of the movie, right? There's a climax coming. Who's the villain? We must know the villain by now. Uh Amazon S3 almost knows it's Sparky status. The database is an event there, and what they do is they create Spark, and they create Spark in what year do they create Spark?
Spark. They release it, version one in 2014. Sure, but sure. Which is surprisingly late. Uh it's peculiarly late, right there. So releasing us like releasing solutions problems that are now almost never almost something to exist, right?
Um anyway. So Datricks uh solves okay, J starts to help solve the problem of how you query a file system. They invent Spark, what Spark does is it allows you to query the file system by lifting more data into memory, by being faster, by being more performant.
What it doesn't do is it doesn't solve the problem that the querying layer needs to know a lot about the data structure. And if we've used PySpark, are people use PySpark and Spark Notebooks and Databricks in this room?
Some people have. And the reason that Spark isn't part of the base path is

because you know that if you're doing that, it's very complicated. Right?
So if you're using if you're using Spark directly, it's quite complicated code. Why do you call it complicated code? Because the code itself has to know about the structure of the data you're working with, um, and you can't use SQL easily.
That's not entirely true. There's some you can massage that, but basically. Now, okay, right. So we need to we need to innovate more. You need to innovate more. Innovate more. So at this point at this point, we have something called um a data lake. What a data lake is just a file system, cloud file system, plus a querying layer.
Cool. Database is this common structure. Got a file data store as a file system. A data lake is a file system plus some previous query system. Cool.
So we answered something that is called lake house. So it's a data lake house. So our journey is local file system. 1960. Database. 1970. Non-relational data. 1990s. Then we repeat the same journey on the cloud. Cloud the file system. Data lake. Cloud. What's the next word? Cloud database to repeat the same pattern.
Cloud database. In other words, what we want is a cloud database. What I mean by that is something on the cloud using a large number of machines that has the properties of a database.
Meaning that it has generalization. So that's what a lake house is. I'll repeat that. Roll written down. I'll repeat that. A data lake house is just a relational database. Running on S3. The cloud data lake house is we've returned all the way back to 1970, and we have a relational database, but now our relational database isn't on like one machine or a small number of machines.
Our relational database is split up between S3, which is where we keep the files, and for example, Databricks, which is where we run the compute.
So it's like having a racial database, a relational database built out of two systems. Cool. Now the key thing for a data lake house is that it's a relational database, meaning it's tabular, and it provides asset or in the OSH.
In other words, it gives you back data quality, making sure data is lasted, it's

consistent, it's a writing real time, etc. And it does so on S3.
Cool. Okay. Anyone know how it does that? What's a file? The way it does that, the way it does that is it uses something called Delta Files. A delta file is like a CS5 file or a spreadsheet or something.
It's kind of just has metadata in it. And it has metadata, in particular logs and other things, that allow it to do that thing of keeping track of where data's being done.
We can provide those countries. Cool. Alright. So uh let me see you at 11am. Oh, don't read the agreement. Yeah. And if we scroll through, there's an interactivity. We've got search fields and places and things.
Hi. Cool. So if on Friday, true capstone, there are senior leaders invited, and if they attend, then you could do that. If they don't. You still can do that. Theoretical possibilities always exist. Yes, you can still do that now. Okay. Right. Uh what are we doing? So I'm going to talk about Docker a little bit and then give you the Docker challenges in the dev container.
Um, which are challenges you have for Docker. And then that's it, then we do for lunch and everything else, right? Um after lunch, we might talk about some other stuff.
We might get started to we're about, I think we're going a bit fast, so really know why. Um miss something from the course, probably. But I think we're going okay. Uh page path, uh DLPC, we've done that. Yeah, just going a bit fast. Okay. DBT. Let me get started on DBT earlier. Okay. Um let's talk about Docker.
And then we'll go into the challenges. So we talk about challenges, by the way, the technical challenges, they can be done individually or in pairs, don't really mind.
However, they're they are uh sort of local up challenges, not so called group 8 discussions or anything. Okay, what's Docker 4? What is Docker? What does it do?
Why do we have Docker? What does it provide? What's its purpose? Containerization. Containerization, right? Yeah, okay. So what is containerization?

So basically you can grab all your dependencies, code everything into one container, and it's probably me. So when we have a piece of software, as you're saying, for example, let's suppose we had some DBT code.
Suppose that, then DBT, for DBT to run, we can pin up various things. You might say, well, don't we just depend on DBT? No, we'd be wrong to say that.
Because we depend on operating system. And in fact, we depend on a table and the cable, right? For DBT to run, the entire universe must be first created, right? That's a doctor sign. So to make an Apple Pi, you must first create the universe.
True. So you've got all these dependencies. Now, containerization solves some of these wrapping up of dependencies.
So you wrap up DBT and you wrap up all the other operating system dependencies you have. Operating system dependences. So for example, you have to have Python installed, maybe for DBT version one, for example.
So you wrap a Python in there. So a container is a bundle of everything. Does it bundle actually does it actually bundle a full operating system?
That's my question. In Docker, do we know we don't bundle from parent system?
No, we don't. So the difference between a container and the virtual machine, you might have heard the phrase virtual machine.
So a virtual machine is similar to a container, but the difference is one has the operating system. One has the operating system, one just pretends to have the operating system.
So when we have a uh a Docker container, if the container's Linux for our machine is Windows, it's still just pretending what it's doing is it's asking Windows to run the things that Linux would otherwise have run.
Now what's the difference there? Well the difference is that with a virtual machine, you're bypassing the host operating system on your laptop, whatever it is, going straight to the CPU, and you're saying to the CPU, run this code.
That means that in some sense, if there's an issue with your Windows operating system or something misconfigure something, that'll just be completely bypassed, right?

So with the container, we're not doing that because it's an extra way. So containers bundle up things together. What else? What other problems does the containers solve?
Repeatable environment. Repeatable environment. So because we wrap up our dependencies into one thing, every time we deploy something, we know it has the same stuff in it.
Yeah. Good. Security. Security, good. So in our world which isn't containerized, we would have a server, and we would have one person's DBT code, maybe another person's, or whatever else, right?
Airflow, we have all these things mixed together, just in files, and we have the programs installed onto the server, and they'd all be running. And if there was code, let's say in some Python script, for example, that read some DBT file or read some airflow file, it would have complete permission to do that.
It's on the same system. In principle, right? Likewise, if DBT needed network access, for example, to day breaks, right? Then anything running on the system would also get the same network access.
Because you give the system network access. So what a container does, a container isn't just a bundling of dependencies. What you also allow a container to do, first of all, is isolate just your just one application per container, VT, and you can control the networking in and out of the container.
So what do we call a channel into the channel here?
The port is a kind of made up really, but it's it's uh a port is an is a message inbox created by the operating system for your application. So every application gets one port or in principle, what a port is associated with one application, so two applications can't share the same port.
So if you put some message in a port inbox, if you like, only one application can read it. So a port, now, okay, so with Docker, you can manage the ports a container has, you can manage what's inside of it.
What else can you manage? You manage the inside, you can manage the networking connection, what else? One final thing really.

RC. I'll take S or H if you like. RAM, exactly. RAM and CPU. SSD or HDD. So why is it useful to be able to restrict a container to run with a certain fixed amount of RAM and fixed amount of CPU?
Why is that useful? So you can restrain it later on. Yeah, so you can restrict it. Yeah, so if there's shared systems or services running, we can cooperate at it. But also, if we just have pairs, if you had a bare description here, um, for example, a website, we could, as I we could attack the website and force the CPU to run to 100%.
Or there could be a bug that would force a CPU to run to 100%. Whereas if we restrict the CPU for a container, even if it thinks its own CPU is running 100%, that's a fraction of the system CPU, so it doesn't hit the system down.
That's a real issue. Okay. So that's it. Now when we uh when we want to uh so when we want to run uh contain when we want to run multiple containers on the system, there's a technology we can use, what the uh configure configure multiple containers to run on a system on one machine.
Here's a container, there's a container, there's a container, there's a container. We can configure them to talk to each other. I Kubernetes, yeah.
Using Kubernetes. That's sometimes written KHS. Or Kubernetes. Kubernetes. And uh what do we call the collection of Docker containers being managed by Kubernetes?
Okay, that's it. Now there's some commands we'll talk about in a second. Talk about Docker file, we'll talk about some commands, then we'll go through that. But um, any questions so far. Okay, so how do we configure how the I've told you that Docker container has in, for example, DBT and has all these tendencies.
How do I configure a single container? In other words, how do I specify what goes in a Docker container? Uh Docker file. Ah, trick question. Docker file specific specify what goes inside a Docker image.
Docker image that's possible. However, that's basically. So basically, what you do is you write Docker file. What Dockerfile does is it defines something called a Docker image.

And an image is a template. It doesn't really matter. So Dockerfile. In the Docker file, we have we write Docker file, we have verbs here in all in capital capital letters.
So we say things like, for example, copy, which copy files into container. At the end of Docker file, we say, for example, run. And we can have other commands like execute commands, you can have port commands.
You have other commands. Well, commands are usually see uh Docker file. Come on. Like update the pseudo. Uh yeah, yeah, yeah. What's that? CMD or run or run.
Hey? Just command line. Yeah, so basically there's lots of words here. There's user. There might be port, you can do to open a port. Uh you can make tag and other things right.
But basically, there's some verbs here in the beginning of the Docker file. At the end of the verb, at the end of the verb here, the action is something that will configure.
So you've got user here, that's what user you're running as. Copy some files into the container with your local system. Copy some files in, copy your dt code in. And the last line of the Docker container is usually run. Because when you run a Docker container, it will run the last line here. User exploits entry point.
Expos, this is why I was thinking for port expos. Entry point. Okay, so we're gonna look at some Docker files. Um that's it. Cool. Let's look at some Docker files and I'll give you the challenges.
So one um interesting Docker file we might want to look at is the Docker file for the dev container. What is a dev container? Dev container is a Docker container that VS Code launches in.
Right. By the end of this course, I'll be a master of interpreting blank faces. So what do I mean by launches in? I mean if I start the terminal here, new terminal. See this command prompt there? That command prompt is running from inside the Docker container. So if there are files I've added, for example, with copy command to the Docker container, they'll be accessible from this command

prompt or CLI.
If there are tools installed inside the container, they'll be available on this CLI.
This command prompt is not the command prompt of my operating system.
If I say, for example, Astro here, oh well, uh me's there's no meas. If I say me's here, unfortunately, some person in Edo has put me's here. Because they're dangerous people, put me's everywhere.
Um the point is that you can see here that this behaves differently than this. This is my operating system, my laptop that is running within the Docker container.
Now the Docker container it's running in. It's called Edo training data products.
That's something that maybe you downloaded as part of the onboarding.
This is provided by Edo. And it's based on another container, which is called Edo data products or something, which is a standard container for creating data products.
So when you create data product on the paved path using paypack knowledge, you will do so from a container that looks a lot like this. And mostly what you'll be doing is writing dbt code in this container from the data products folder.
So here's a product folder, and you will write maybe some DBT code. Can I get some code, please? Any code somewhere? Um not really what I want. I think this is this is the starter code for the some challenges.
Do we have do we have any solutions here? We do have solutions somewhere. I'll show you some DBD code anyway. That's uh these are uh those are challenges.
Where's the solutions? Can you get some can I get some solutions, please?
Could I have a solution? Uh okay, there's some DBT code. You can see it uh I'll get you some more exciting code later on when I figure out what folder we're actually going to be in.
But you can see it's like SQL, right? So mostly what you do when you create a data product on the page path is you get the uh data product container from GitHub and you create some dbt files, and then you run dbt, you can run DPD from the container.
What that will do is run it on day bricks, it'll create the tables in the background,

it'll do all the work, that's it. Or you can write some airflow code in the container, and then you can commit your git code from within the container, you say git commit or something, and that will cause all the nice machinery to work and someone's gonna bend a repo and you're gonna get their own file.
Yeah. Yep, yep, yep, yep. I'm gonna have Martin uh talk about this on Thursday morning, maybe, or tomorrow morning, whenever it turns up, you're gonna turn up at some point.
Right? So we'll turn around, or we'll open our eyes and spot Martin will be here. Right? Um and he's gonna talk through the um the page path technologies and explain their motivation so that it is the design motivation for why CG chose them. It's also gonna go through the data product container in a lot more detail and say, here's where the standard folder layout is, here's some YAML files you want to create, etc.
And I could do that, but Martin is, you know, you can then ask Martin questions about CG stuff. Why haven't we done it this way, or about Dremio, or about laser, all these other things, which uh I should not speak about because every time I speak about something, people need to be specific about CG policy.
Martin can think about CG policy and the relationship to other systems, etc. Right, so here's going to do that. Um okay, what do we anyway? Where are we?
Where are we? What are we doing? Well, um what we're doing is we're looking at the dev container, and I said we're gonna do two things. One look at the Docker file of the dev container.
So this is the file that defines the environment that IDO have provided to run your code. You don't have to learn this or worry about this. This is up to Edo to manage.
I'm just showing you it. So we've got user command, we've got run command. What are we doing with these run commands? We're running some installation scripts.
We're copying things into the container. We're setting some operating system environmental variables, which are configuration variables for the operating

system.
And we have a run command of the red one. Now there's one important line here that we haven't talked about, and then we can talk about that, then we do the challenges.
Which is the first line from. What is from doing there? Importing. Sorry? Importing. Importing Kyle, yeah. Importing what? Base image good, like a word in there.
So see how we see we've got the word from let's look at line two there. From CG registry, blah blah blah blah blah. Python 3.13. That line there refers to another Docker file.
Basically. That has configuration in it for working with Python 313 in a certain environment. So basically we're extending, so the rest of the Docker file extends from that line three template.
Almost all Docker files start with a from line, basically all of them. Because they are usually just configuring some other standardized template.
Most Docker files are much simpler than this. This is a very complicated one.
So the answer is no. This is just the content of the container. To configure, we can use Kubernetes and things, which we're not going to talk about too much, but um, you've got Docker Desktop.
So this is where you have your images listed, the templates that you've created. Um do some configuration here. Um there's also some settings here, resources. Um this this reason this is resource application for all containers. But then when you when you run a container from the command line, you can do different things there.
Um what I'm gonna give you now anyway, is in your challenges folder. Your challenges folder might look a little bit different from mine, but it has this structure.
There's a Docker Docker intro here. And in the Docker intro, there you can go, you could open preview. Or you could um, for example, edit it. And there's some challenges for working with Docker.

So for example, check Docker version, um, run a container, uh, look inside a container, we'll create a Docker file, create a new image, create a container from the image, and play around.
Cool? So it's like a Docker 101. So I'm gonna give you until 1 pm to do that. Now many now, probably about a third of you, if not half of you, I don't know I don't know, 40% of you kind of know this stuff already.
If you do know this stuff already, feel free to, for example, look at some more advanced Docker stuff that you don't know. So what about building, for example, multiple containers from one Docker file?
What about um Kubernetes stuff that you don't know? What about I don't know, up to you. Get the idea? If you're already an expert in Docker, you can probably go and learn some more stuff.
So go and do that. But for everyone else, um, work through the challenges, and uh uh you've got how we've got 35 minutes of that before lunch. Lunch is 12 p.m. And I'll see everyone back here at 1 p.m. Cool. Feel free to use AI to explain things if you don't know what the what's going on to help. Feel free to use catalyst to generate some more examples as you wish.
So we can go in there report, right? We can go now. You should, I think on the on in your onboarding you, yeah, you should have already done that now.
Delete it, clone it, delete it, clone it. Yeah.
If we need someone from the platform seam to come and support us, we can get we can probably get more.
Yeah.
Should we do something that aims to be fun? Yeah. That would be fun. Maybe. Well, we're I would say we're about two hours ahead of the course. Up to me, we'd be done.
If it was up to me. There would be uh a huge bowl of whiskey sour. Some sangria for the all the people who don't drink whiskey sour. Picnic back picnic blanket. Do you know what a scotch egg is? So you take a hard-boiled egg and you wrap it in sausage and bread crumb it, and then you deep fry it. And you serve it cold.

We would have that. And we would have sausage drawers. Sausage and pastry? So you take sausage mints. You want the sausage mints the inside of sausage sausages?
Take these sausage mints and you wrap it in the pastry. And you bake it. You have a sausage drawer. You would have a mints pie. Do you want means pies? Short crust pastry, not shortcrust pastry is like pie pastry. Could I put beef mints in the middle of that? What else would we have? Eh? I know a guy can make a call.
You're a bit disappointing. An American would have had to make it. I wouldn't even I wouldn't even trust someone from the South of India to make it, let alone America.
So I am from. Which is the far north of England near Scotland. Although if you cross into Scotland, that's when you start deep frying strange things.
So because we're ahead, uh, what we can do is we so I want you to think about maybe uh I think Martine's coming tomorrow morning, and that sounds about right to me.
Um so Martin likes to go through the paved path. You might show up repos and configuration files, YAML stuff, uh, more CG specific stuff. Um ahead of that, it would be good to understand the things that he will be talking about.
That makes sense, like to do a preview in a bit more in a bit more detail rather than the full detail. So I thought, well, let's go through DVT a little bit, let's go through Airflow a little bit more, and astronomy a bit more and Databricks a little bit more, so that when he talks tomorrow, you'll be prepared.
I'm gonna be fighting digestion, right? And I fear I might lose. So what we can do is we can just sort of do it in a different way. So here's a different way.
Uh first of all. Is it recording? First of all, is it recording? Because every time uh I've closed laptop. I said there's two failure modes of this course.
One, transcript isn't turned on, two, I've closed the laptop. Remember, the whole point of this course is the transcript. Anyway. Anyway. What's my point?
My point is that I thought if I told if I talk to you for too long just about stuff, you

might get bored. Or uh digestion would win. So what we can do is maybe um come up with some exercises, group things, challenges, individual activities.
So I thought, why don't you, for example, take your data contract that you have and ask catalyst to generate DVT code for it? Why don't you ask it to generate airflow code that matches the SLA that you have in your contract?
Why don't you then, for example, ask Copilot or whatever, to add code comments throughout the files to explain what the code does? Why don't you ask AI to explain how it runs?
How do I run this? Right? So rather than me teach you Airflow and DBT, you generate things, you teach it yourself using AI, then we come back, I'll mup up the chaos.
I'll make sure you get toy anyway, because you're getting taught it as part of Wednesday and Thursday. So we're not gonna I'm gonna leave you with just half fragmented knowledge, right?
But maybe it might be more fun to start with the contract, generate some stuff, work backwards from that, understand how it works. Yeah? So in terms of generating stuff, are we just generating the content in the code, or are we actually executing and going somewhere?
Um so first of all, let me show you. So what I was thinking of is how far can you get by yourself starting to do a contract generating stuff? So I'm happy for you to go all the way into pressing play and if you can do that.
Because then we can do a little um kind of group debrief. And we can say, what were the commands that you run on the CLI to run DVT code? What would what was the airflow commands?
What is it what does the airflow syntax do, right? How do I run something daily in Airflow for example? Maybe someone can tell me. Maybe someone can't, right? However, that's what I'm saying is you can go as far as you like. But let me show you Daybricks. So hopefully on the onboarding um document, you saw a mease command.
Me's run setup call on Databricks, I believe. Yes. And you you just press enter

until something useful happens, right? Enter enter enter. Fingerprint.
Come on. Compute computers are very fast. Extremely fast. In a second on a computer, how many operations can I run in a second? How many calculations could I run on this computer in a second, do you think?
Any guesses? Yeah, many billions, yeah. So we've got we got four point four, so four point six billion. It's actually about double that. It's about eight billion.
And then I have ten cores. So that's eight billion times ten, so that's eighty billion in a second. That's a lot. How many do you think are needed?
To do what we've just done there. Me logging into um logging in. Clean the back more processes. Yep, calling all of it. Oh, a tiny fraction of eighty billion.
So why did it take such a long time? What have they done to my beautiful computer? We gave you this. And you've ruined it. What have you done to it?
Network a cable. Yes. You've plugged network cables into this beautiful machine.
So just to log into something now, you have to go to twelve places across the globe, right?
That's what's low. So if you have the experience that computers are really slow and annoying, it's because some adversary of you, the one who hates you, has connected everything you do to a com to a um to uh a goddamn internet connection.
So even if I type in Apple notes, if I just type in Apple nodes, just one letter. M. Where's that going? You see what I'm saying? I mean that's been stored in the cloud, so it'd be backup, all this.
Crazy. Crazy. Anyway. The point is, once you've got that set up, if I keep pressing enter, what that will do is it will connect your this environment here to the Databricks environment that you have.
And let me bring up the URL. So it might give you a URL here somewhere. Um if it doesn't. Databricks. I think it's this environment here. Okay. So if you look, if you want to, if you want if you want proof that it's fast, right?
Everything I'm doing on my computer, everything I'm doing in my computer, which includes running Docker containers, their entire simulations of other

machines.
Everything I'm doing is 10%. So 90%. The CPU can run 80 billion calculations in a second, roughly. So in a second, it's um nine, you know, in a second, 90% of its time is doing nothing.
Ruined it all of you. You know the end of um the end of uh what's that one with the monkeys, the apes at the end? We know where the with the Statue of Liberty. What's that? What's up moving? Oh. Uh no, it's an old movie, older than that. Planet of the Apes, that's kind of the we know any of that you you you've ruined it, you've ruined the earth.
There you go. Anyway, now we've got now it's open, I can continue my conversation. Right, so if we look in this is Daybricks, you can see this, right? And if we go to not go to work, supposed to get the catalog. Um if we go to training dev, in training dev, you should have a training underscore.
I have a training underscore, for example, con M H A B. And whenever I run DBT code, which you'll later discover yourself if you're playing around, um, you will create uh tables in this folder, basically.
It's not folder, it's a schema, but whatever. So here are my dbt tables. There's no sample data there, but maybe there's some sample data elsewhere.
Anyway. The point is that if you do me's setup databricks properly and you authenticate properly, then all the dbt stuff you run will create stuff here for you. So you actually can actually open this, see this, and correspond it to what you're doing in DBT. Cool. I'll put I'll post a link to this in um in the Teams if you want it. But it also it is also given to you when you set the thing up, but I I I don't exactly have there. Uh co at 7 IU. Yeah. Okay, there you go. There's a there's a link to that.
Thank you. There's a link to that um databricks. So let's just let's just pause for a second on Databricks. Now I know you know not I know that many of you, if not all of you, or almost all of you, know nothing about DBT.
How is a DBT creating anything, right? That's tomorrow. I wanted you to see how much you can find out today, otherwise it'd be bit boring, right?

Any case, um what I will say a few things about Databricks. Just while we're here. So this is the Databricks UI. One of the things that people tell me at the end of this course is they assumed they would be using the Databricks UI more than they do.
There's not really a lot of reason to use the Databricks UI if you're on the paved path. Because you're not creating notebooks, you're not you're not going to Databricks to create a notebook or anything, like you would do if you're doing Spark and the people in the room who do that.
Um why? Because you're writing DBT code in files in this environment. That's it. And then you commit the code to Git and it's all packaged up for you, and we'll see Martin will go through the pipeline now.
However, you're not really using the Databricks UI that much. Not least that Catalyst should in fact be able to search Databricks for data sets as well, search the inter catalogue.
So in the case, what we're seeing here is the catalog, uh Databricks catalog. And um this is where all of the datasets on Caspian eventually should be.
And data sets on certainly on the page path will all be here. And there's gonna be different catalogues for different business domains. And again, Martin can talk about that.
If we can talk, you know, there's source line catalogs and domain line catalogs and all kinds of catalogs. You can search for data in. So in the future, hopefully. Uh all the data you need will be in the catalog, and you probably can browse through it and find it. Well, if there's a catalog. Um, what is it to say?
That's it. So actually, most of the Databricks UI you won't even you don't even need to use that much. It's just a compute backend for running DBT.
Alright, so what I want you to do, as I said, just to repeat myself, you've got a data contract. Ask catalyst, generate me a DBT project or generate me DBT code based on my data contract.
So if there's no data, you could say simulate some data. Right? Um, according to the schemas in the contract, create the DBT code. Then you're gonna say, add

comments to the DBT code so I understand how it works.
You're gonna ask catalyst all these questions, how does it run? How do we get this running? Where will it be when I go to the makes sense? And then you do the same for airflow.
For example, you would say, I want to set up an airflow job so that this is creating the consumer aligned data set every day or every hour or something, right?
Or it's it's creating when there's new data coming in, isn't it? Then you ask it to create the airflow code, ask it to add comments to every line, explain the airflow code, and then we're all done, we'll have a little debrief and we can see what we've learned.
Cool. If you know all these things already, you can go and do more advanced stuff. Arbitrarily advanced stuff. For example, if you know the DBT.
Do you know how many people know DBT already? One person. Have you ever created DBT macro? Yes. Uh do you know how the bi-temporal loading macro works?
If you do. Oh. Can you think of a bi-temporal loading strategy that isn't covered by the bi-temporal loading macro? I think you can. I think that could be your task.
Um good. Uh who knows uh Airflow? Anyone in the Airflow already? Yeah. Okay. You use it. So you could um have you do you know about collecting airflow to data dog and doing logging from AFO into data log?
Oh, it sucks. Sucks. You know what you're gonna do. You know what you might investigate right now, how do you improve that? Uh how do you improve that? How do you improve the user experience? Is it like a little library we can do import not subset of data or something? There's the powerful job. But yeah.
Okay, but if you know everything already. I'm I'm I'm giving you if you know DBT already, you can maybe go down the DBT macro route and create some contacts macro, is that helpful for everyone if you know F4 already got down the how do I improve the after experience with people?
In any case, most of you don't, as you've nominated. So I just want you to play around. Cool. Anyway, why don't we come back together at like something like

quarter past two?

1. Give you like an hour to play around. Cool. Alright.

I forget you don't use 24 hour plots. I only use 24 hour clocks. Can I turn the projector off? You don't really need this one, do you? No. Somebody's sitting back there and just put one.

You can go there if you want.

Um I think we can go at 230.30 rather than three. Depends, is the transcript running? Is it right? One of them is running. Well, the good one. It's not running now.

But thank you for reminding me, because I don't know what the D I don't know what the debrief is going to be. So that's a look. Thank you. Good.

But we but given that, you know. 3pm is our usual finish time, so I don't I don't mind it. If we're if we're head of time, that's fine. Um okay, what are we doing?

So debrief. So what is DBT? Anyone want to tell me what DBT is?

So frame or platform. Yeah. So version one's written in Python, yeah. What does DBT do? More as like a cloud. Project manager? Good, could be, yeah.

Data build tool. It kind of probably. Yeah, it says ETL. Does it want people what ATL is? Yeah. Do you want ELT is?

Yeah. So it's really an ELT tool because what happens with DBT is you load, you seed data into the roots first. That's the last kind of tracking load, and then you do transformations in data.

Transformations happen less. Okay. So what when you're writing so DBT provides ELT, so that means it extracts loads and transforms data. Right. When you're writing DBT, what do you what are you looking at?

What are you actually writing? What do the code look like? Mostly SQL. What else is in it? It's not just SQL, it's something else. There's YML files as well, the templates, right?

The double the double braces. What are they for? What do they do? Yeah. They were they they replace things with things between the braces are replaced by other pieces of SQL.

The whole thing in the end is SQL. Cool. So all of the SQL you've written so far begins with what word? Wall of the SQL, all of the SQL. It's like.
So if it's an ETL tool, which means the tool for loading, which means insert, right? And create and extracting and loading, which means create. If it's a tool for inserting and creating data and creating tables, why do we only see select queries?
Create implicitly maybe building the. So the select queries are actually translated to create table and insert. So when you so what dbt does when you write an effect query, a field is a select query defining the fields of the create table.
And the rows that the query is selecting are the ins are inserting to that table. So you're writing the select query, but actually creating a create table and an insert.
Cool. Or maybe. So old DBA here. So when you all of the lovely innards that us old school DBAs, like indexing all the lovely. That's OLTP though, isn't it?
Yes. Do we know the difference between OLTP and OLAP? So OLTP is about writing data to databases. It's about that linearization guarantees transactions. Anyone know why? So in a relational database, we often have these things called transactions, right? Which are these uh units of query where we can bundle up several queries together that potentially mutate the database and have them all succeed or fail at the same time.
Yes? So you can add and remove at the same time. Anyone know why that's called transaction? What is the word transaction background? Pardon? Because we can roll there.
So the word transaction comes from retail transaction because the original um relational databases were retail databases in like shops or stores.
So you have a customer transaction. And the the first thing you need when you're dealing with money, for example, is you need database transactions because you want the ledger to be correct, right?
You want to debit from one account credit to another account. Right? So the the primitive, the most ex the most basic and original form of a TWA-based transaction was a recall transaction.

Debit from one account credit to another account. There is one organization in the world, in the well in the world. There's one organization in most countries that does not need to handle money using transactions.
What organization is that? The central bank. So they can create money without corresponding. In fact, that's how central banks work, right? They create money first, then they go to the bond market, pretend they haven't created money, and then buy bonds, and then they try and pretend that the accounts are settled. Anyway. Someone goes, well, if you're creating money all the time, why do you even need bonds? A dangerous question to ask. Anyway. Well, why do you need a financial industry?
The answer is uh pensions and rich people. That's the answer. So the what so why why does government funded state? Why does government fund positive, why does the government fund negative central bank balances with bond issuances?
The answer is just pen. It doesn't have to. Just pensions and stuff. Anyway, what would happen if we didn't have a bond market? Nothing. Actually, no.
Okay, anyway, that's some side points about how central banking works. Let's go back to DBT. Um, so in DBT, we've got SQL queries which are translated into create and insert.
And because we're in an OLAP world, OLAP world, which means that we're in a read and reading world, analysis world, which we're bulb inserting and bulk reading.
We care less about that kind of optimization. Now the OLAP world, I would agree with you, is a crazy world. Doesn't make any sense. I think it's slow, nothing's annoying.
I don't know. I just an analysis of data sets is uh for some reason the data systems that support that are you know you're doing bulk inserting of large amounts of data, it's just slow, isn't it?
It's a slow annoying world compared to the OLTP world of transactions where I think it's fast. Anyway, so we're in DBT and we have SQL queries, they're being

translated into create and insert statements, good.
What else can DBT do? What what are the um see any test dbt testing that goes on? Quality tests, you know? The YAML files, what's in the YAML files, what are the YAML files for?
We'll look at we'll look at tests on when we'll look at test tomorrow because don't get all that. What are what are the aml files? What's inside the YAML files?
What do they do? They they handle the tests, they provide some form of testing, yeah. They also can change the materialization of your queries. From view to table.
So for example, do anyone know what what what material what is materialization mean? Anyone want to find materialization? What's in between create table and create view?
Yeah, so so you can have queries be virtual, in other words, they're just held in memory, or you can have queries be creating tables. So when dbt translates your SQL to create, it can be create table or create view.
Now you would probably want your source tables to be tables, and your curated or your output tables to be tables. Your intermediate might might just be a view. tables to be tables and your curated or your output tables be tables your intermediate might might just be a view because they're an intermediate right so it depends if an it depends if there's another downstream consumer of those intermediate intermediate tables whether you actually want them to be a table or a view.
So you can configure the materialization in the YAML with the only VR. Okay. Um what's what what CLI commands do we use to run DBT? What are the commands that we've seen so far run test build?
Debug debug? DBT debug, good tells you you've got connections subject C what is C do? That's not a big take some local data, for example in a CSV file and uploads it into Databricks.
Why is C the common operation do you think? No, because most of it's going to be in Databricks or any S trace could be your morning style or your Bloomberg or

whatever or your internal data sets, right?
So you're not going to be seeing that much it's useful for uh learning. Um okay that'll do for DBT we can we talk about DBT is not um airfould anyone generate any working airflow code some people data cycle business that's fine what is airflow for question good.
How do you define airflow what what are we defining? In DBT we define something called a model and dbt language that's a neutral it means either a table or view or something else.
So DBT, our SQL files they define something called model could be a table, could be a view could be et cetera, right? And the name of the DBT file is the name of the table or view.
In Airflow, what are we defining in airflow? We're not defining models we're defining class. Okay could be yeah let's call it DAG FO calls a DAG and what is DAG stand for?
Directed acyclic graph which is another word for basically tree and it's a tree of what tasks different tasks yeah do one task and then another task and another task it's very simple straight line it also have branches do a one task or another task also have fan out of words you can have one task that starts then many tasks in parallel when you are defining so a DAG is basically a task collection.
If I was writing if I had a written F I would have called the DAG I would call it a task sequence of test collection task tree. That's why I would call it but they want you to vendorize your mind didn't they steal your thoughts and vendorise them.
So you don't on Amazon you do you don't call a virtual machine a virtual machine you call it EC2 right? You don't call it task sequence you could tell you could take that away and talk to someone about that right?
You call it a DAG. They don't want you taking away anything vendorized mind. Or turns and mush and barely think anymore. EC2 RD50 SQS Lambda function.
What's a lambda function? It's a script that's why I would call it Amazon scripts okay so airflow uh what kind what what is a language we write airflow tasks in?
Python Python we'll kim to an example tomorrow hopefully my team will talk

about it where we can write them in YAML in a thing called blueprints what you can do is you can you can create uh airflow tasks in Python and then leave placeholders if you like and then use YAML just to provide the placeholders roughly now the reason you do that is you have to work on the many of your tasks are the same just create some standard templates and then your data creators only need to say well this is my table this is where I'm going etc. Because your actual tasks you're doing each time are not very different. Cool? Okay. Um does anyone know the operator I'll just use the word operator that connects Airflow to DBTabricks operator.
Does the Databricks operator so if you want to run a Databricks notebook you can say databricks do something right there's also dbt task group or GTS.
So if you generate some airflow code generating airflow code let's say it's run some dbt code what you'll see in the airflow code is it's saying dbt something like dbt task group dbt task what that what dbt task group is is it's a piece of Python code Python function or something along those lines that if you like represents your dbt pipeline as airflow casks.
So when you run an airflow DAG each phase of the DAG is a phase of your dbt project cool we'll look at that on Thursday as well in more detail what else do people find out that I haven't covered what is uh yeah okay well yeah what else do people find out can we can we have some more stuff can we have three minutes of new information is that okay can I beg what else do people find out when they were create stuff?
Could be anything. Doesn't have to be about dbt or a flow. What prompts did you use? Anyone use some prompts an elephant gun with some diet coke well come on can we get some prompts?
Did you ask about these things? Go on? Okay. Was getting created in the data you saw the table getting created in databricks everyone see their tables getting created in Databricks that's reassuring isn't it that tells you something's happening that makes you a data product creator by the way that's what data product creator is someone who can get tables created in databricks.

How well did your data contracts translate to DBT? Did anyone have any issues? What kind of issues just translate very straightforwardly pretty well actually I thought it was interesting does again I had you said AI do a big chunk of it modeled two different types of data sets because I had internal products and external products created two separate models created two separate tables and then created a view to union them all together.
Well I didn't tell it to do that just decided to you could you you hand crafted with with the AI model the data contract right? Right. So you did in some way well verbally we did when we discussed it in the you know so there you are you you know so your thoughts were there um cool did anyone iterate on the dbt code that was generated or the FO code and we iterate what were the iteration you did?
So it was just it was just good straight away. Perfect so one way of viewing this course is just what are the things that you should think about when babysitting catalyst right what other things we should this is why we do design meetings why we do the data contracts because those become even more important if that's what's going into catalyst to generate all this stuff isn't it?
So some of you might think I know we've had some technical extreme technical people in the course that go well why am I going to learn about blah blah blah blah blah right things that are design things right?
Well if 90% of the technical job, well not let's say 80% technical job is done in the first pass from the data contract let's say then getting the data contract right getting the PRD right etc.
right etc doing those interviews recording those interviews getting the transcript right iterating with in conversation that's like 80% of the the product right the other 20% is I've generated the code I'm running the code it isn't quite right I then need to go in and iterate and so on I need to review the code make sure the tests are correct have we missed any edge cases etc but actually quite a lot of the human creativity the original thinking the the new information that isn't in Copilot or whatever that comes from design discussions cool so that's why we

spend a good day I'd probably spend the day the first day doing that right first day in the morning and then reviewing the SETMs now what I do these days myself is I have a nice development loop what I do is I transcribe dump into AI and then I actually print out uh paper and I remember this print this I print it out and I go over and I hand I handwrite editing it so if you're if you're converting some of those data contracts to HTML or something and you may you could say make it black and white you could say make it other these other things I don't know but then you can maybe even print it out you can maybe even sit down and go this is five pages it's a data product if it's my job to make one right it's your job so go and go make it well right to go and print it out and then sit down and read through it.

Over lunch you know the coffee or something you go well this is wrong this is wrong. I find it a little difficult to as you say you know these pages and pages of markdown I find it a little difficult to sort of go through that and understand what I need to be doing.

So another thing I do is I I put the transcript on when I'm working through it. So I'll say for example page 40 work example morning star exposures there's a leak there's a reference to RFD 42 I don't know what that is context decision consequences those look like titles those should be in title font how can a creator participate in early design decisions this seems trivial.

Maybe you move it to a footnote that's going to be in courseware but if you were doing data contracts you would say for example um these columns don't look quite right we need to go and search this you say pull that transcript right then iterate that again iterate again print it out and then once you've got the design recommendation down that's going to raise you to 80% of the code I mean I don't suggest printing the code out because the code's relatively small compared to the design documentation right cool just sit in silence for two minutes.

No you can go you can get out leave you've you've you've crossed the line now I've had it up to here with you all